06 树

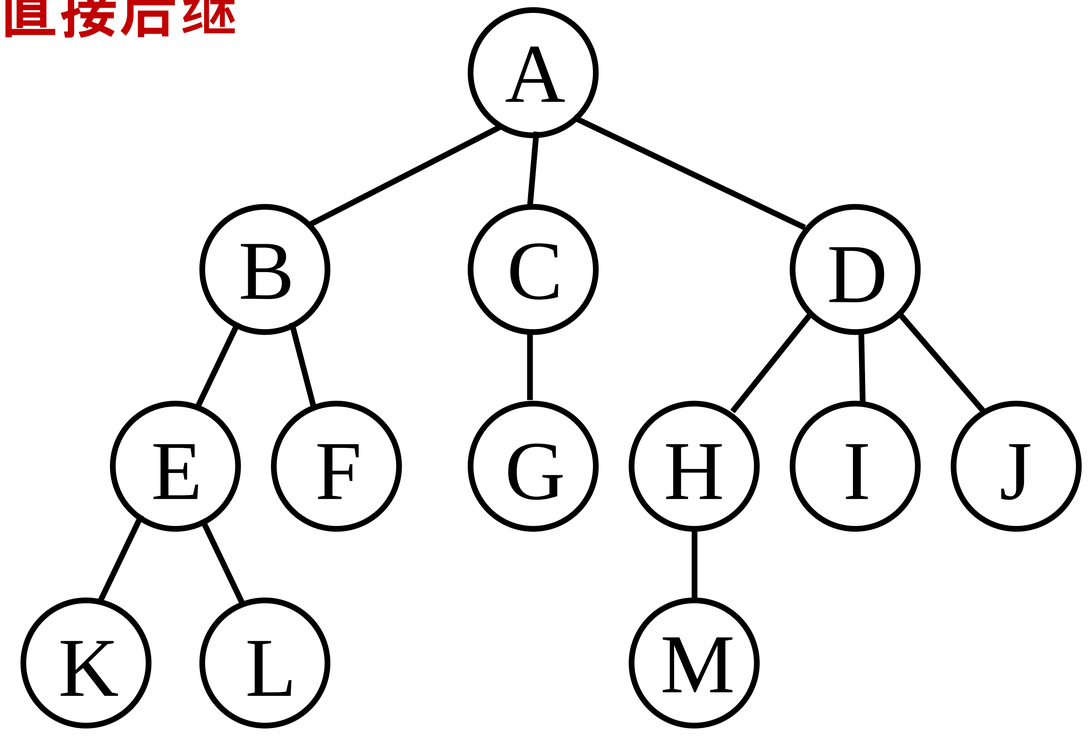
- 树
- 二叉树

06 树

- 树
- 二叉树

树的定义

- 树(tree)是由0个或多个结点组成的有限集合T
  - 有一个特定的结点，称之为根结点(root)
  - 其余的结点分成m(m≥0)个互不相交的有限集合$T_0, T_1, \ldots, T_{m-1}$，每个集合又都是一棵树，称$T_0, T_1, \ldots, T_{m-1}$为根结点的子树
  - 特点：除根结点外每个结点有且仅有一个直接前驱，但可以有0个或多个直接后继



树的基本概念

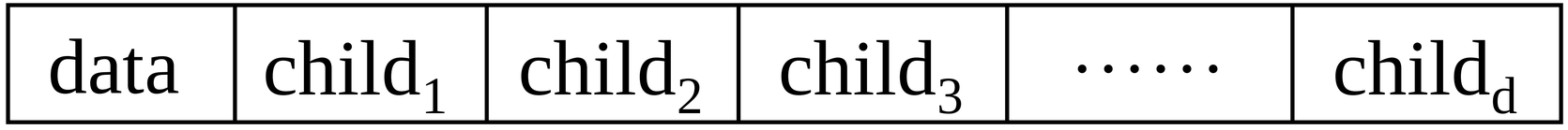
- 结点的度(degree)：一个结点的子树个数
- 叶子结点(leaf)：度为0的结点，即叶子结点没有子树
- 子女结点(child)：若结点x有子树，则子树的根结点就是x的子女结点，也叫子结点
- 双亲结点(parent)：若结点x有子女结点，则x就是其子女结点的双亲结点，也叫父结点
- 兄弟结点(sibling)：同一父结点的子结点互为兄弟结点
- 祖先结点(ancestor)：根结点到结点x的路径上的所有结点(但不包括x本身)称为x的祖先结点
- 子孙结点(descendant)：结点x的子结点、以及这些子结点的子结点都是x的子孙结点(但不包括x本身)

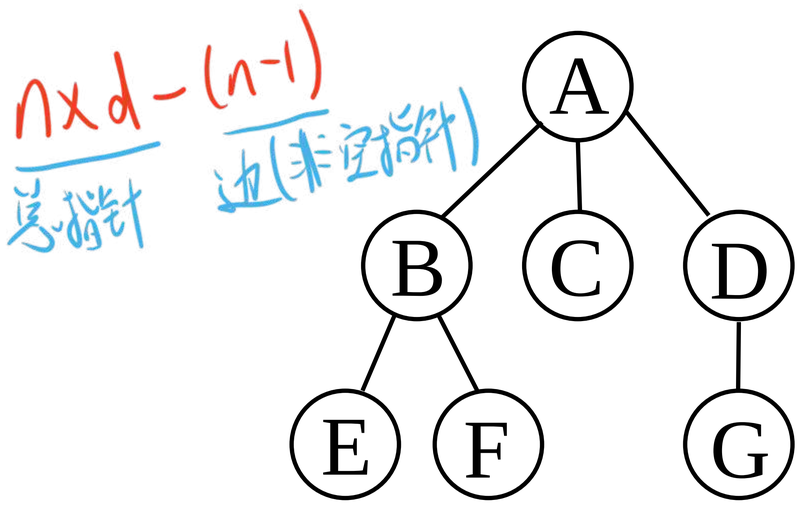
树的基本概念

- 结点的层次(level)：树的根结点所在的层次为1，而其它结点的层次等于其父结点的层次加1
- 树的深度(depth)：树中层次最大的结点的层次
- 树的高度(height)：树中层次最大的结点的层次
- 树的度(degree)：树中结点的度的最大值
- 有序树(ordered tree)：树中结点的各子树$T_0, T_1, \ldots$是有次序的，其中T_0为根的第1棵子树，T_1为根的第2棵子树
- 无序树：树中结点的各子树之间的次序是不重要的，可以互相交换位置
- 森林(forest)：有限棵树的集合

树的存储结构

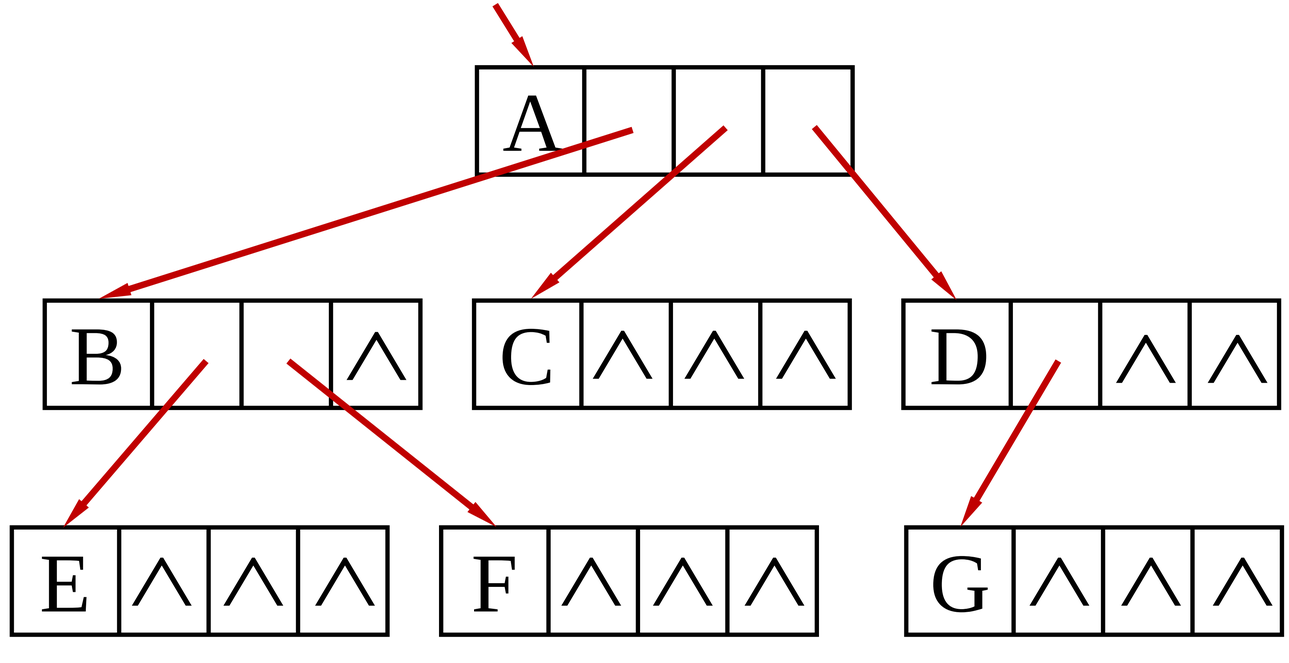
- 一种直观的存储表示
  - 一个结点用两个部分表示：数据部分存放结点的数据，指针部分存放指向各个子结点的指针
  - 可以根据树的度d为每个结点设置d个指针域





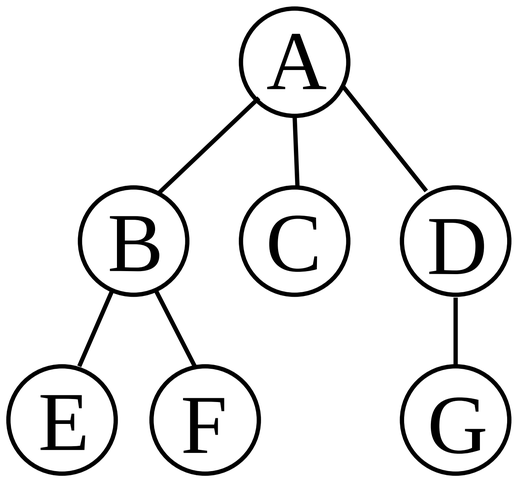
n×d-(n-1)
总指针
边(非空指针)

2n+1个空指针域



树的存储结构

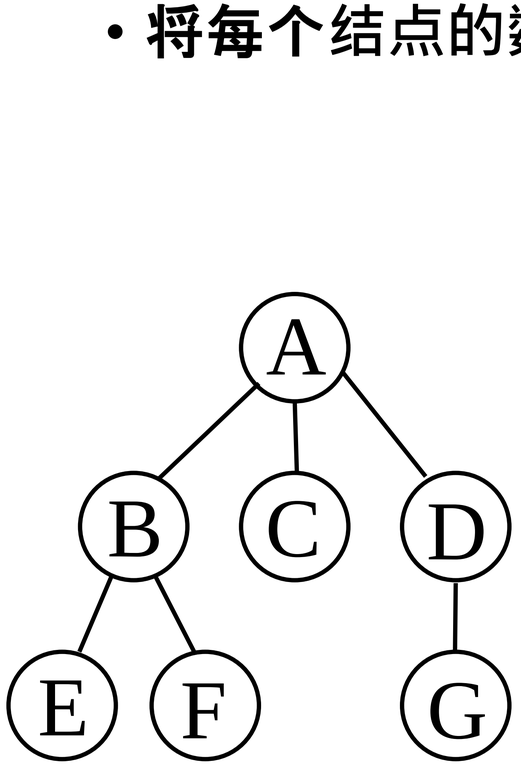
- **父指针表示法**
  - **每个结点有两个域，一个存放结点数据，另一个是存放指示其父结点位置的指针**

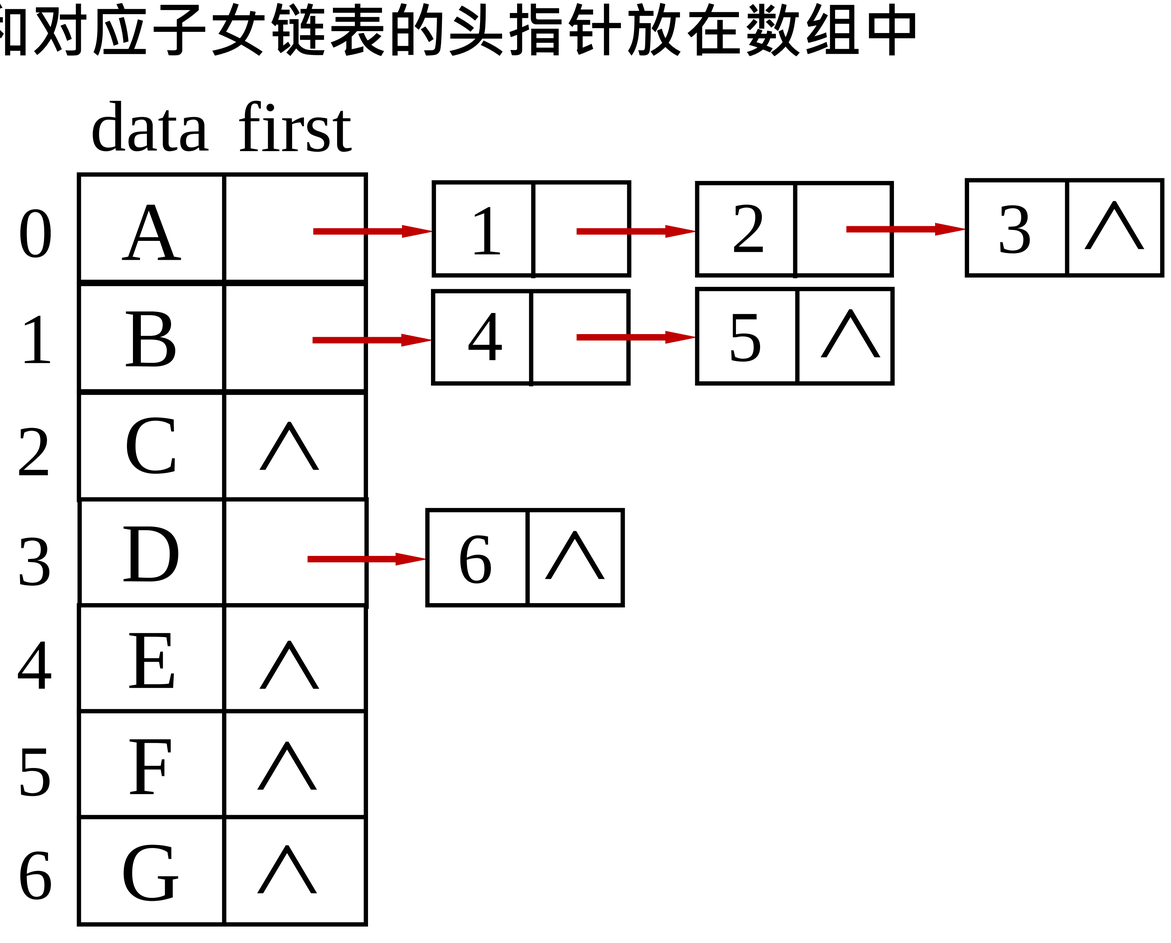


	0	1	2	3	4	5	6
data	A	B	C	D	E	F	G
parent	-1	0	0	0	1	1	3

树的存储结构

- **子女链表表示法**
  - 为树中每个结点设置一个子女链表
  - 将每个结点的数据和对应子女链表的头指针放在数组中

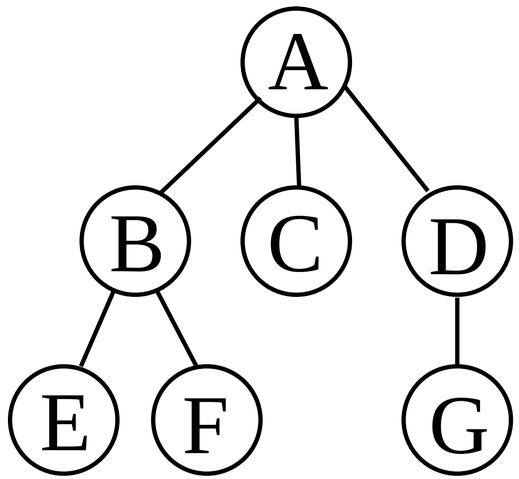


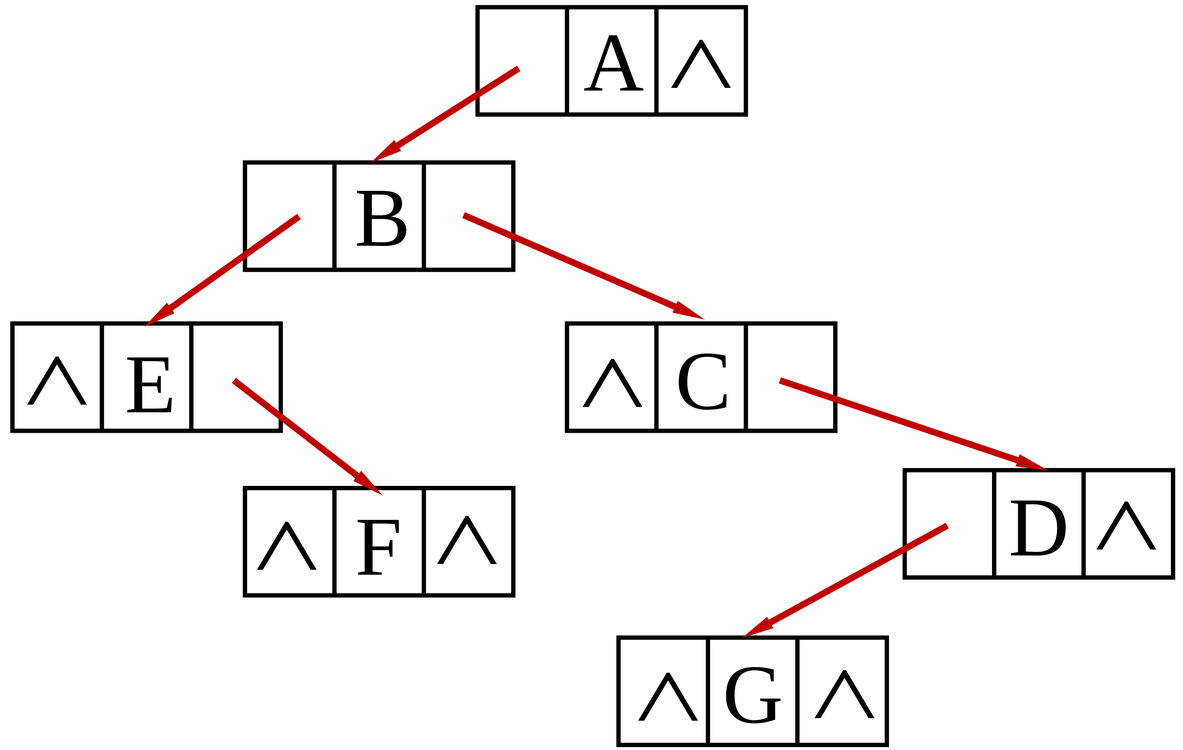


树的存储结构

- **左子女-右兄弟表示法**
  - **每个结点有三个域，分别存放结点数据、结点的第一个子女的结点地址、结点的下一个兄弟的结点地址**

data	firstChild	nextSibling





左子女-右兄弟表示的树的类定义

```
template <class Type> class Tree;
template <class Type> class TreeNode {
    friend class <Type> Tree;
    private:
        Type data;
        TreeNode <Type> *firstChild, *nextSibling;
    public:
        TreeNode(Type value=0,
          TreeNode <Type> *fc=NULL,
          TreeNode <Type> *ns=NULL) :
            data(value), firstChild(fc), nextSibling(ns) { }
};
```

左子女-右兄弟表示的树的类定义

```
template <class Type> class Tree {
  private:
    TreeNode <Type> *root, *current;  // 根指针及当前指针
    // 在以p为根结点的子树中搜索值为value的结点
    bool Find(TreeNode <Type> *p, Type value);
    // 删除以p为根结点的子树
    void RemoveSubTree(TreeNode <Type> *p);
    // 在以t为根结点的子树中搜索p的父结点
    int FindParent(TreeNode <Type> *t, TreeNode <Type> *p);
```

左子女-右兄弟表示的树的类定义

```
public:
    Tree( ) { root = current = NULL; }
    bool Root( );  // 让树的根结点成为树的当前结点
    bool IsEmpty( ) { return root == NULL; }
    bool FirstChild( );  // 寻找当前结点的第一个子结点
    bool NextSibling( );  // 寻找当前结点的下一个兄弟结点
    bool Parent( );  // 寻找当前结点的父结点
    bool Find(Type value);  // 在树中搜索值为value的结点
    void InsertChild(Type value);  // 在当前结点下插入子结点
    bool RemoveChild(int i);  // 删除当前结点的第i个子女
    ……
};
```

左子女-右兄弟表示的树的实现

```
// 让树的根结点成为树的当前结点
template <class Type> bool Tree <Type> :: Root( ) {
    if (root == NULL) { current = NULL;  return false; }
    else { current = root;  return true; }
}
```

左子女-右兄弟表示的树的实现

```
// 在树中找当前结点的双亲，使之成为当前结点
template <class Type> bool Tree <Type> :: Parent( ) {
   // 空树或根为当前结点，返回false
   if (current == NULL || current == root) { current = NULL;  return false; }
   TreeNode <Type> *p = current;
   return FindParent(root, p);  // 从根开始找p的父结点
}
template <class Type> bool Tree <Type> ::
FindParent(TreeNode <Type> *t, TreeNode <Type> *p) {
   TreeNode <Type> *q = t->firstChild;
   while (q != NULL && q != p) {  // 循根的长子的兄弟链，递归搜索
      if ( FindParent(q, p) )  return true;
      q = q->nextSibling;
   }
   if (q != NULL && q == p) { current = t;  return true; }
   else return false;  // 未找到双亲，当前结点不变
}
```

左子女-右兄弟表示的树的实现

```
// 在树中找当前结点的长子，成为当前结点
template <class Type> bool Tree <Type> :: FirstChild( ) {
    if (current != NULL && current->firstChild != NULL) {
        current = current->firstChild;  return true;
    }
    current = NULL;  return false;
}

// 在树中找当前结点的兄弟，成为当前结点
template <class Type> bool Tree <Type> :: NextSibling( ) {
    if (current != NULL && current->nextSibling != NULL) {
        current = current->nextSibling;  return true;
    }
    current = NULL;  return false;
}
```

左子女-右兄弟表示的树的实现

```
template <class Type> bool Tree <Type> :: Find(Type value) {
    if (IsEmpty( ))  return false;
    return Find(root, value);
}
// 在根为p的子树中搜索值为value的结点，成为当前结点
template <class Type> bool Tree <Type> ::
Find(TreeNode <Type> *p, Type value) {
    bool result = false;
    if (p->data == value) { result = true;  current = p; }  // 搜索成功
    else {
        TreeNode <Type> *q = p->firstChild;
        while (q != NULL && !(result = Find(q, value)))
            q = q->nextSibling;
    }
    return result;
}
```

左子女-右兄弟表示的树的实现

```
// 在当前结点下插入值为value的子结点，若当前结点没有子结点，
// 则新结点成为当前结点的第一个子女，否则成为最后子女
template <class Type> void Tree <Type> :: InsertChild(Type  value) {
    TreeNode <Type> *newNode = new TreeNode <Type> (value);
    if (current->firstChild == NULL)
        current->firstChild = newNode;
    else {  // 当前结点有子女时
        // 找当前结点第一个子女
        TreeNode <Type> *p = current->firstChild;
        while (p->nextSibling != NULL) // 扫描兄弟链，找最后子女
            p = p->nextSibling;
        p->nextSibling = newNode;  // 新结点成为其最后子女
}
```

左子女-右兄弟表示的树的实现

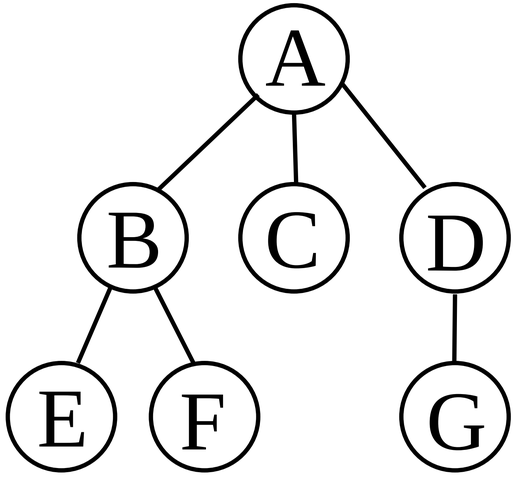
```
// 删除树中当前结点的第i个子女及其全部子孙结点
template <class Type> bool Tree <Type> :: RemoveChild(int i) {
    TreeNode <Type> *s;
    if (i == 1) {  // 要删的是第一个子女
        s = current->firstChild;  // 要删的结点是s
        if (s != NULL)  current->firstChild = s->nextSibling;
        else return false;  // 无子女时退出
    } else {  // 要删的不是第一个子女
        TreeNode <Type> *q = current->firstChild;  int k = 1;
        while (q != NULL && k < i-1) { q = q->nextSibling;  k++; }
        if (q != NULL)  {  // 找到第i-1个子结点
            s = q->nextSibling;  // 要删的结点是s
            if (s != NULL)  q->nextSibling = s->nextSibling;
            else return false;  // 无第i个子女时退出
        } else return false;  // 链中结点个数少于i时退出
    }  RemoveSubTree(s);  return true;  // 删除以s为根的子树
}
```

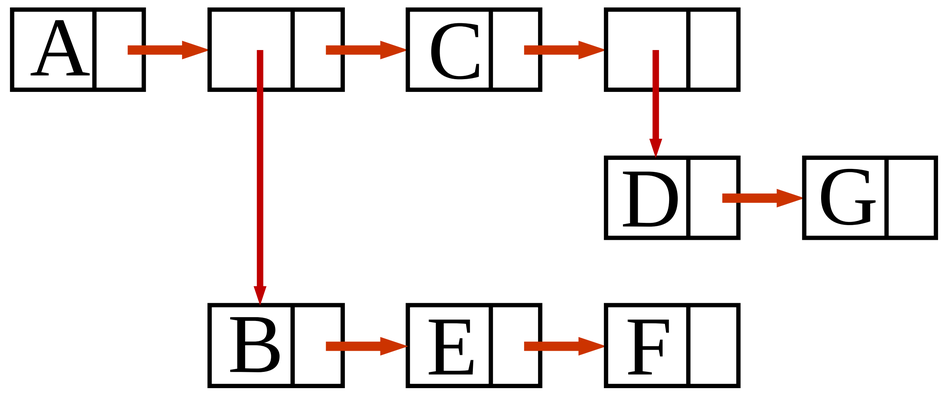
左子女-右兄弟表示的树的实现

```
// 删除以p为根结点的子树
template <class Type> void Tree <Type> ::
RemoveSubTree(TreeNode <Type> *p) {
    // 找p的第一个子女q
    TreeNode <Type> *q = p->firstChild, *next;
    while (q != NULL)  {
        next = q->nextSibling;  // next为下一个子女
        RemoveSubTree(q);  // 删除以q为根的子树
        q = next;
    }
    delete p;
}
```

树的存储结构

- 广义表表示法
  - 叶子结点->原子结点
  - 非叶子结点->子表结点
  - 根结点->表头结点



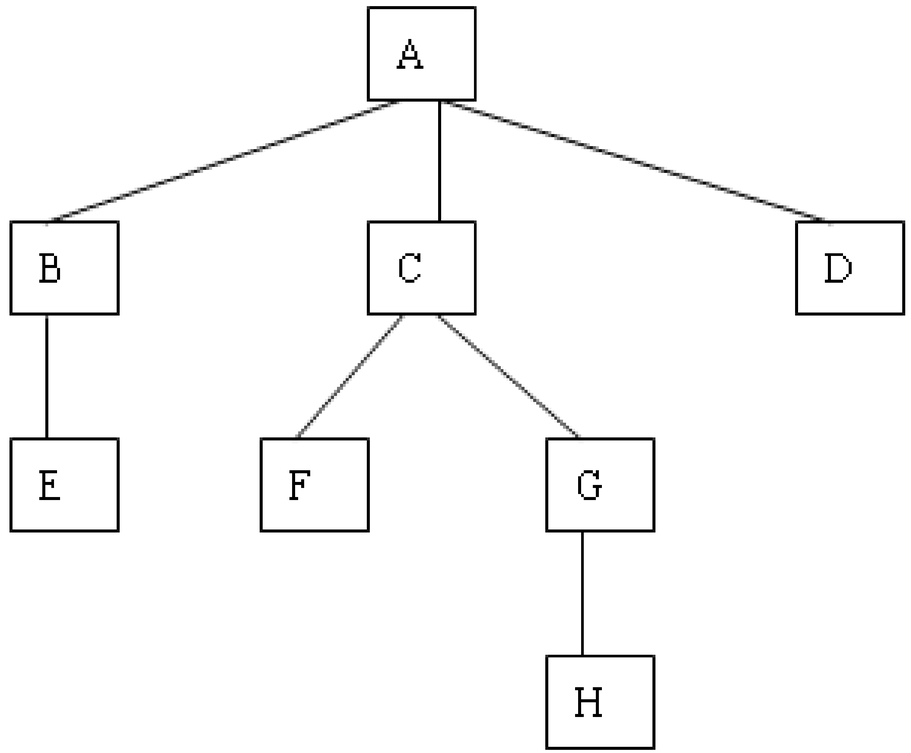


A(B(E, F), C, D(G))

树的遍历

- 树的遍历是按一定顺序访问树中每个结点，同时每个结点只能被访问一次
- 有多种遍历方式
  - 深度优先遍历：前序(先根)遍历、后序(后根)遍历
  - 广度优先遍历：层次遍历
- 对于有序树，每一种遍历得到的结点序列都是唯一的
- 对于无序树，原则上一个结点的子结点之间是无序的
  - 但在具体程序中，一般是按一个结点的子结点表中实际的存储顺序遍历这些子结点

树的遍历



- 前序遍历
  - ABECFGHD
- 后序遍历
  - EBFHGCDA
- 层次遍历
  - ABCDEFGH

树的前序遍历

- **前序遍历：先访问根结点，再依次前序遍历根的各子树**

```
template <class Type> void Tree <Type> ::
PreOrder(TreeNode <Type> *p) {
    if (p != NULL) {
        cout << p->data << endl;  // 访问根结点
        for (p = p->firstChild; p != NULL; p = p->nextSibling)
            PreOrder(p);  // 递归前序遍历子树
    }
}
```

树的前序遍历(非递归)

```
template <class Type> void Tree <Type> ::
PreOrder(TreeNode <Type> *p) {
    Stack <TreeNode <Type> *> st(DefaultSize);
    do {
        while (p != NULL) {   // 沿长子链向下
            cout << p->data << endl;  // 访问根结点
            st.Push(p);  // 进栈
            p = p->firstChild;
        }  // 走到无左子女结点跳出循环
        while (p == NULL && !st.IsEmpty( )) {  // 无子女或无兄弟
            st.Pop(&p); // p无子女但可能有兄弟，或p无兄弟
            p = p->nextSibling;  // 退栈，转向p的兄弟，或p的父亲的兄弟
        }
    } while (p != NULL);
}
```

树的后序遍历

- **后序遍历：先依次后序遍历根的各子树，再访问根结点**

```
template <class Type> void Tree <Type> ::
PostOrder(TreeNode <Type> *p) {
    if (p != NULL) {
        TreeNode <Type> *q;
        for (q = p->firstChild; q != NULL; q = q->nextSibling)
            PostOrder(q);  // 递归后序遍历子树
        cout << p->data << endl;  // 访问根结点
    }
}
```

树的后序遍历(非递归)

```
template <class Type> void Tree <Type> ::
PreOrder(TreeNode <Type> *p) {
    Stack <TreeNode <Type> *> st(DefaultSize);
    do {
        while (p != NULL) {   // 沿长子链向下
            st.Push(p);  // 进栈
            p  = p->firstChild;
        } // 走到无左子女结点跳出循环
        while (p == NULL && !st.IsEmpty( )) {  // 无子女或无兄弟
            st.Pop(&p);
            cout << p->data << endl;  // 访问根结点
            p = p->nextSibling;  // 退栈，转向下一兄弟
        }
    } while (p != NULL);
}
```

树的层次遍历

- 层次遍历：先访问第0层的结点，再访问第1层的结点，再访问第2层的结点，再依次访问以下各层的结点

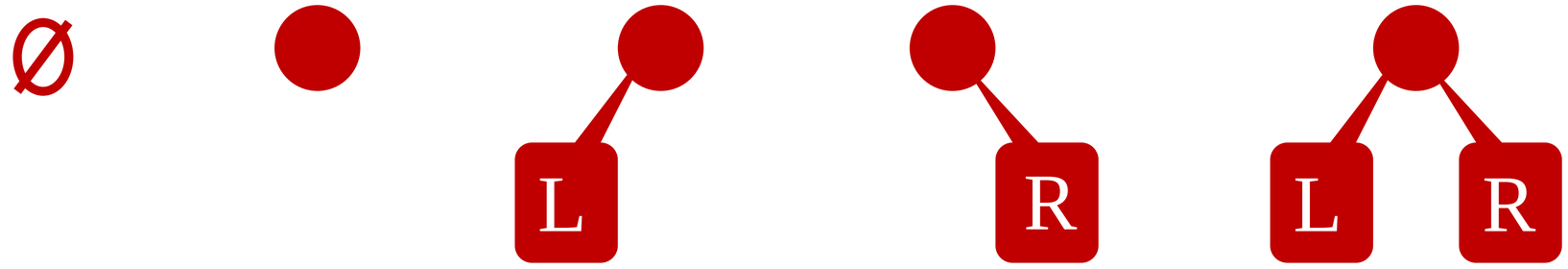
```
template <class Type> void Tree <Type> ::
LevelOrder(TreeNode <Type> *p) {
    Queue<TreeNode <Type> *> Q(DefaultSize);
    if (p != NULL) {
        Q.EnQueue(p);
        while (!Q.IsEmpty()) {
            Q.DeQueue(&p);
            cout << p->data << endl;
            for (p = p->firstChild; p != NULL; p = p->nextSibling)
                Q.EnQueue(p);
        }
    }
}
```

06 树

- 树
- **二叉树**

二叉树的定义

- **二叉树(binary tree)是一个有限的结点集合T**
  - **T或者为空，或者由一个根结点及表示根结点的左、右子树的两个互不相交的结点集合所组成，根结点的左、右子树也是二叉树**
  - **特点：每个结点最多有两个子结点，即二叉树中不存在度大于2的结点，且二叉树的子树有左右之分，其子树次序不能颠倒**



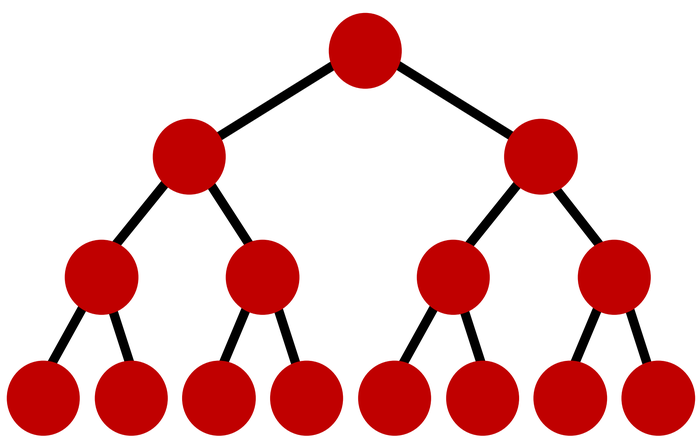
二叉树的五种不同形态

二叉树的性质

- **性质1**：若二叉树的层次从1开始，则在二叉树的第i ($i \geq 1$)层最多有2^{i-1}个结点
  - 数学归纳法证明
- **性质2**：高度为h的二叉树最多有2^h-1个结点($h \geq 0$)
  - 等比级数列求和证明 $1+2^1+2^2+\cdots+2^{h-1}$
  - 最少有h个结点
- **性质3**：对任何一棵二叉树，如果其叶结点（度为0）有n_0个，度为2的非叶结点有n_2个，则有$n_0 = n_2 + 1$
  - 设度为1的结点有n_1个，则总结点数$n = n_0 + n_1 + n_2$，总边数$e = n_1 + 2n_2 = n - 1$，因此$n_1 + 2n_2 = n_0 + n_1 + n_2 - 1$，即$n_0 = n_2 + 1$

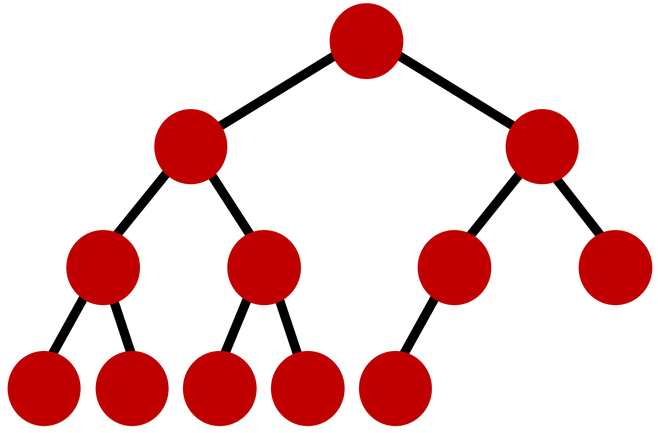
满二叉树

- 满二叉树(full binary tree)是一类特殊的二叉树，其每一层结点数目都达到了最大
- 除最底层结点的度为0外，其它各层结点的度都为2，因此高度为h的满二叉树有$2^h - 1$个结点



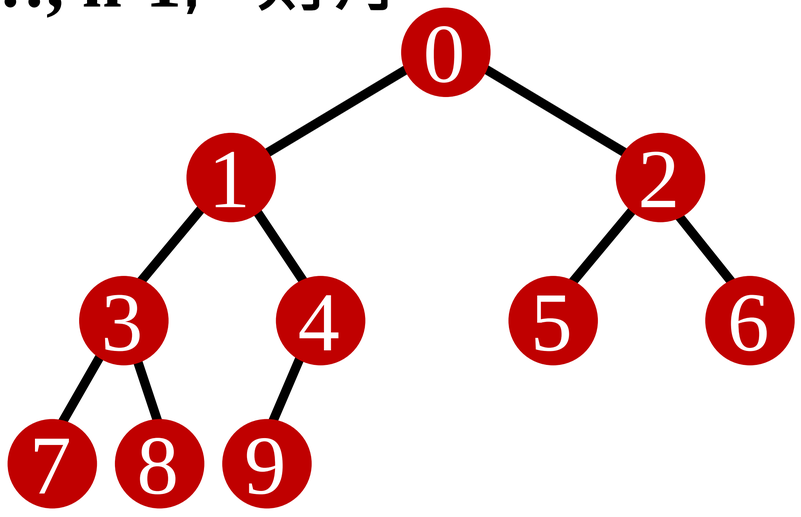
完全二叉树

- 对于一棵二叉树，**除最后一层外，其它各层的结点个数都达到最大，最后一层则从右向左连续缺0个或者多个结点，这就是完全二叉树(complete binary tree)**



完全二叉树的性质

- **性质4**：具有n (n≥0)个结点的完全二叉树的高度为$\lceil \log_2(n+1) \rceil$
  - 设完全二叉树的高度为k，则$2^{k-1}-1 < n \le 2^k-1$，即$2^{k-1} < n+1 \le 2^k$，两边取对数有$k-1 < \log_2(n+1) \le k$，即$k = \lceil \log_2(n+1) \rceil$
- **性质5**：将一棵有n个结点的完全二叉树自顶向下，同一层自左向右连续给结点编号0, 1, 2, …, n-1，则有
  - 若i=0，则结点i无父结点
  - 若i>0，则结点i的父结点为$\lfloor (i-1)/2 \rfloor$
  - 若2*i+1 < n, 则结点i的左子女为2*i+1
  - 若2*i+2 < n, 则结点i的右子女为2*i+2
  - 若i为偶数且i != 0，则其左兄弟为i-1
  - 若i为奇数且i != n-1，则其右兄弟为i+1

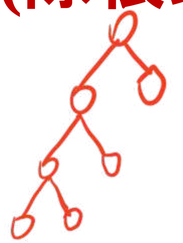


练习1

- 一棵完全二叉树上由1001个结点，叶结点有多少个？
- 由二叉树性质得$n_0=n_2+1$，且完全二叉树的$n_1=0$或1；已知二叉树的总结点数$n=n_0+n_1+n_2$，则有$n=2n_0+n_1-1$；代入得$1001=2n_0+n_1-1$，因1001为奇数，故$n_1=0$，得$n_0=501$

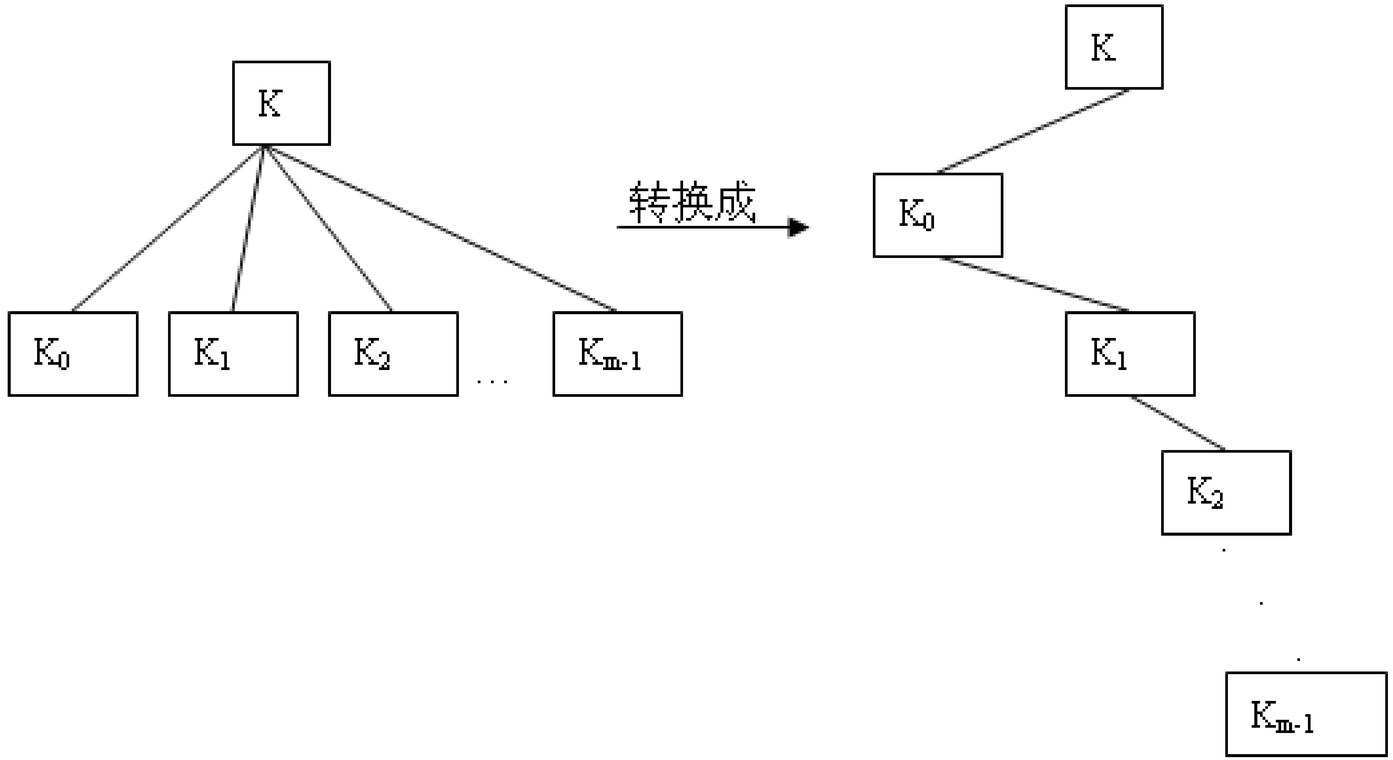
- 一棵完全二叉树有124个叶结点，最多有多少个结点？
- 由二叉树性质得$n_0=n_2+1$，则$n_2=123$，故最多有$124+123+1=248$个结点

- 设深度为k的二叉树上只有度为0和度为2的结点，则这类二叉树上所含的结点总数最少为多少？最多为多少？
- 最少为2k-1(除根结点每层2个)，最多为2^k-1(满二叉树)

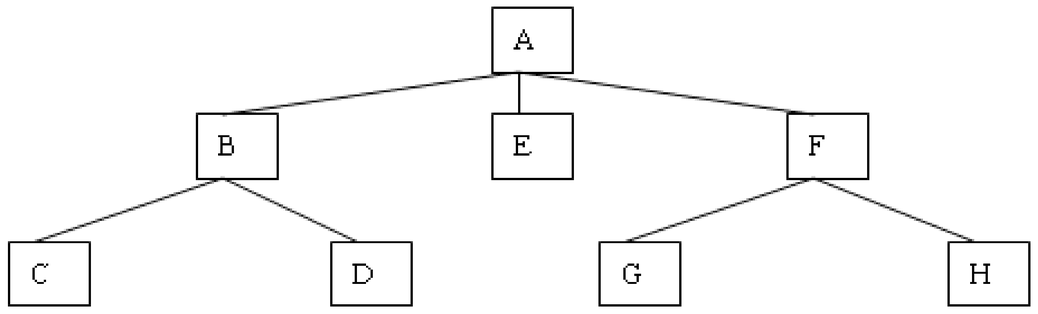


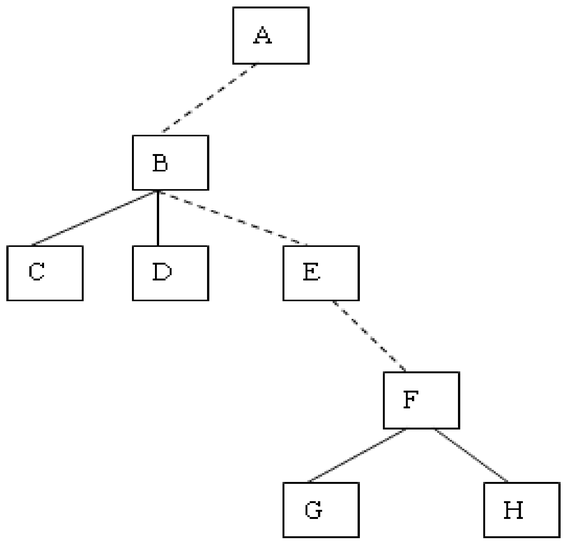
任意次树转化成二叉树

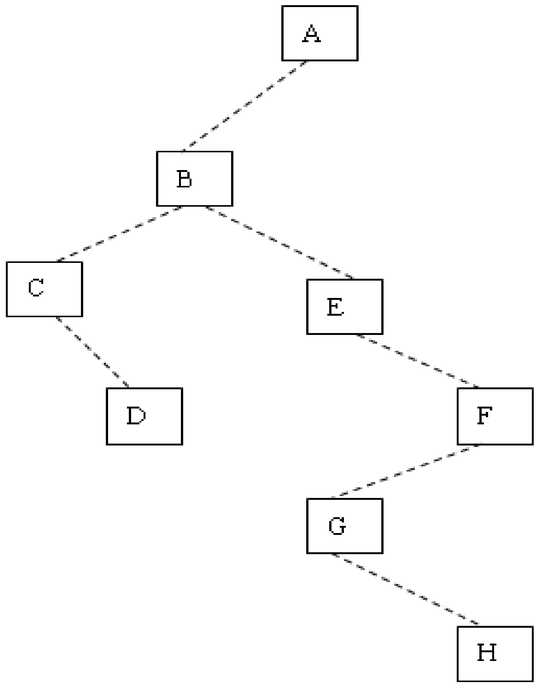
- **把具有m个子结点$k_0, k_1, \ldots, k_{m-1}$的结点k转换成以$k_0$作为结点k的左子结点，并且$k_{i+1}$作为$k_i$(i=0, 1, …, m-2)的右子结点，这种方法叫做左子女右兄弟法**
  - 树的前序遍历顺序与对应二叉树的前序遍历顺序相同
  - 树的后序遍历顺序与对应二叉树的中序遍历顺序相同



任意次树转化成二叉树的过程示例



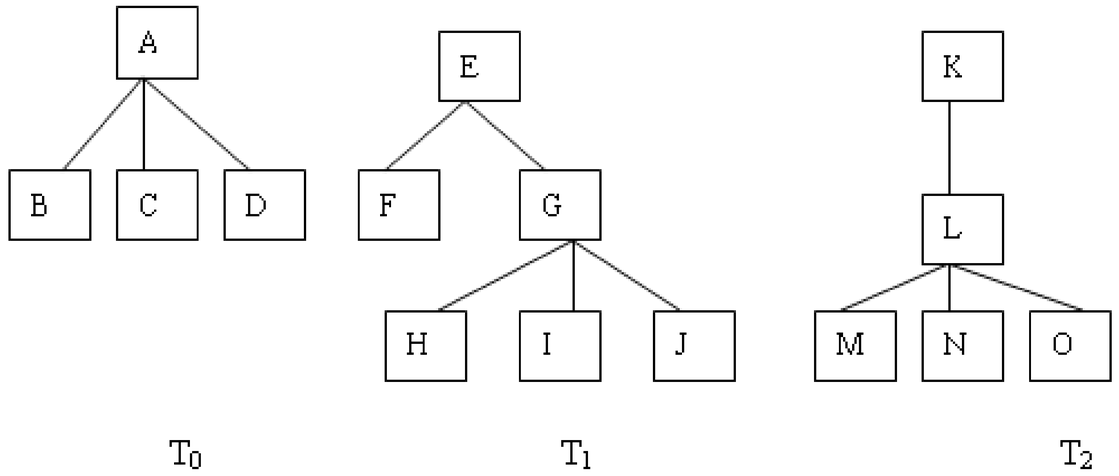


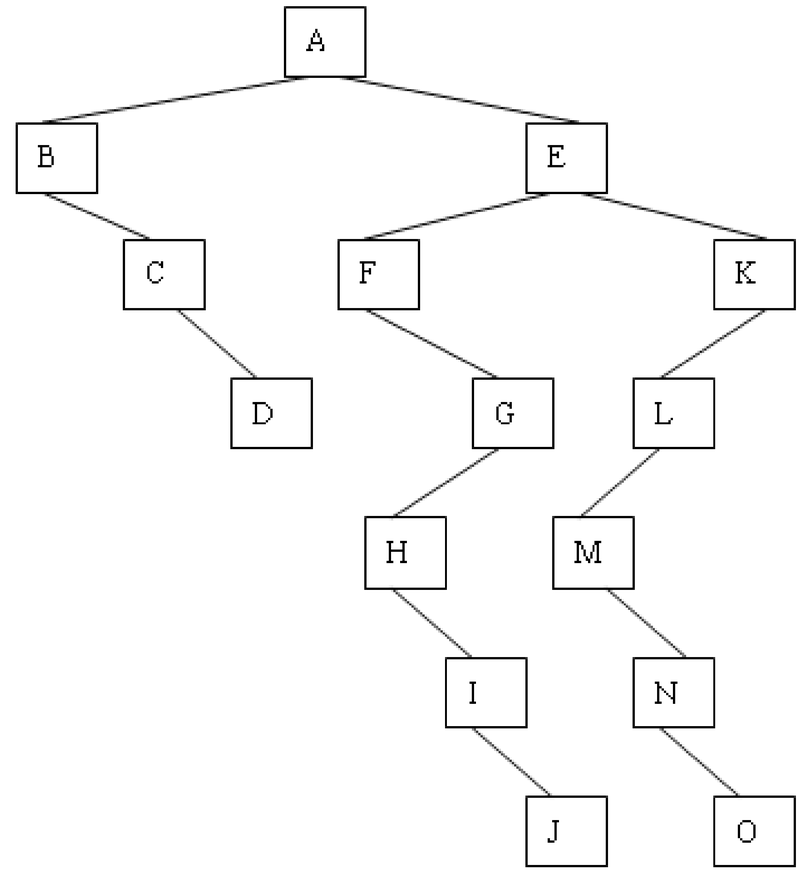


森林转化为二叉树

- **T = (T_0, T_1, …, T_{m-1})是由m(m≥0)棵树构成的森林，得到对应二叉树β(T)的方法如下**
  - **如果m=0，那么β(T)为空二叉树**
  - **如果m>0，那么β(T)的根结点就是T_0的根结点，β(T)的根结点的左子树是β(A_0, A_1, …, A_{r-1})，其中A_0, A_1, …, A_{r-1}是T_0的根结点的子树，β(T)的根结点的右子树是β(T_1, …, T_{m-1})**

森林转化为二叉树示例



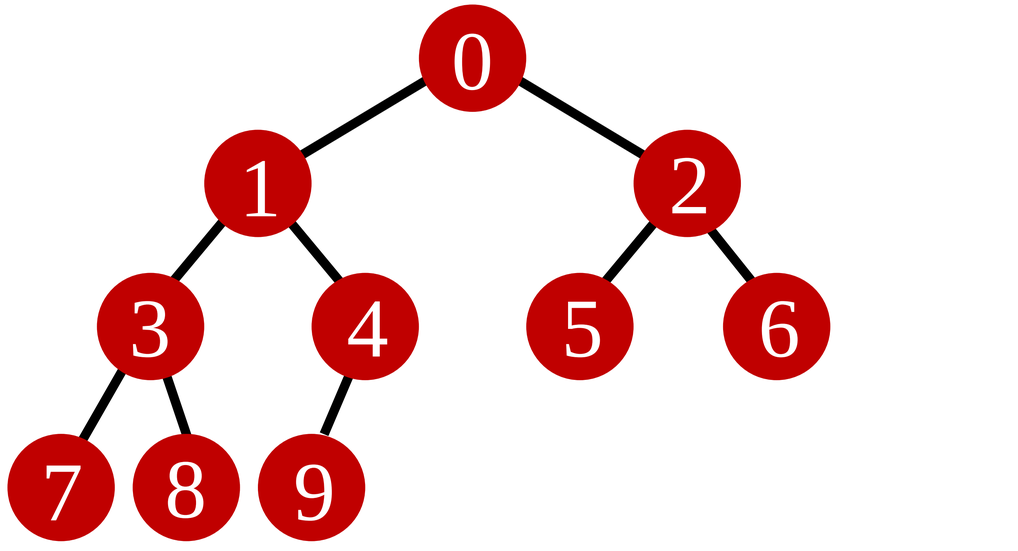


二叉树转化为森林

- 如果二叉树为空，对应的森林也为空
- 如果二叉树非空，则对应的森林中
  - 第一棵树的根为二叉树的根
  - 第一棵树的子树森林是由二叉树的根的左子树转化而来，森林中其余的树组成的森林是由二叉树的根的右子树转化而来

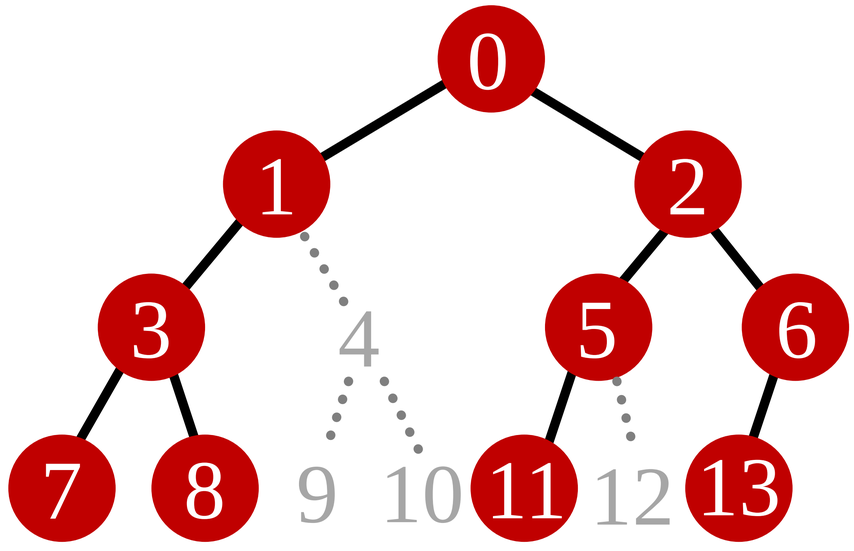
二叉树的存储表示：数组

- 对**完全二叉树**的所有结点按照层次次序自顶向下，同一层自左向右遍历，得到一个结点的线性序列，按该序列将二叉树放在一维数组中
- 利用完全二叉树的**性质5**，从一个结点的编号推算出其双亲、子女、兄弟等结点的编号，找到这些结点



二叉树的存储表示：数组

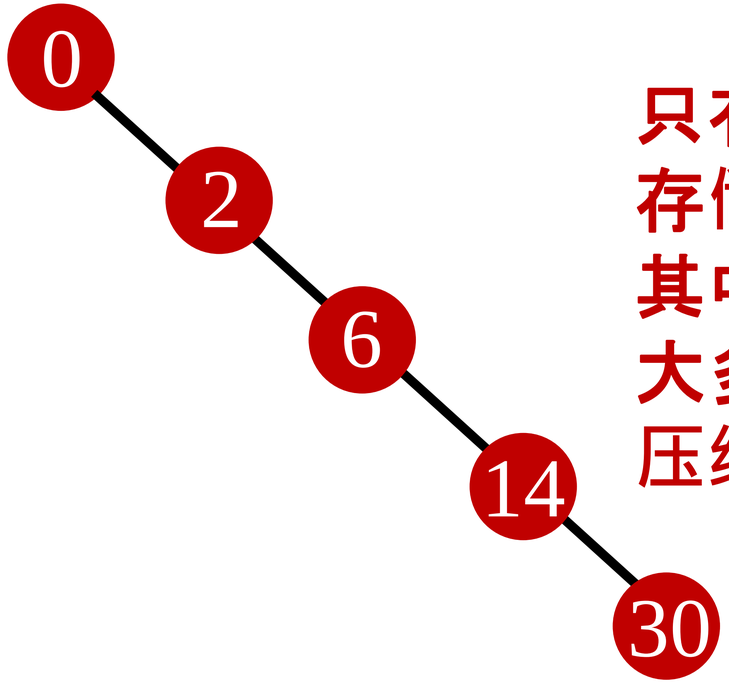
- 对**一般二叉树**的结点按照层次遍历次序进行编号，然后按其编号将它放到一维数组中
- 在编号时，若遇到空子树，应在编号时假定有此子树进行编号，而在顺序存储时留出相应位置，由此找到其双亲、子女、兄弟结点的位置，但**可能消耗大量空间**



0	1	2	3		5	6	7	8			11		13

二叉树的存储表示：数组

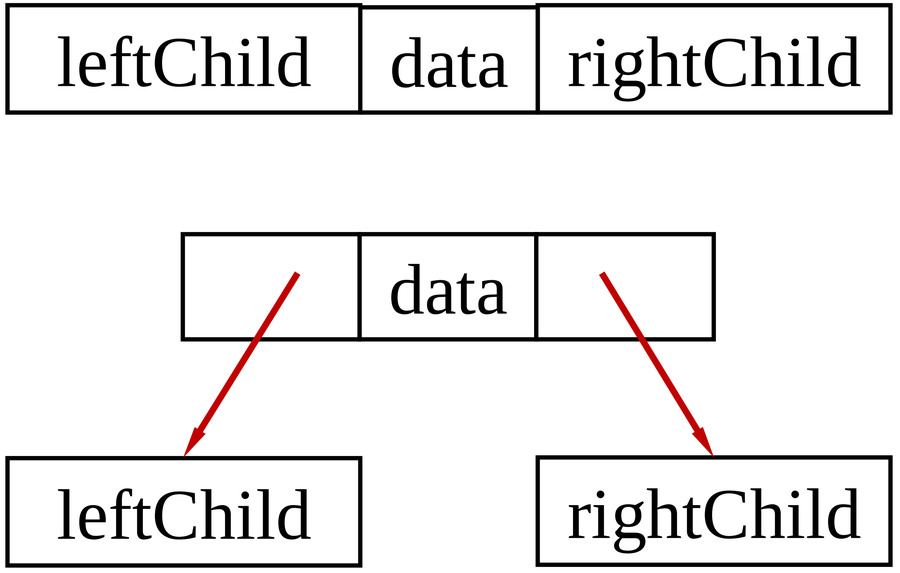
- 可能会有较多的空间浪费
- 删除或者插入树上的结点时，被删除或者插入的结点的子结点的层次会发生变化，需要大量移动数组中的元素



只有右单支的二叉树：要求一个可存储31个结点的一维数组，但只在其中5个位置放有结点数据，其它大多数结点空间都未利用，又不能压缩到一起，造成很大空间浪费

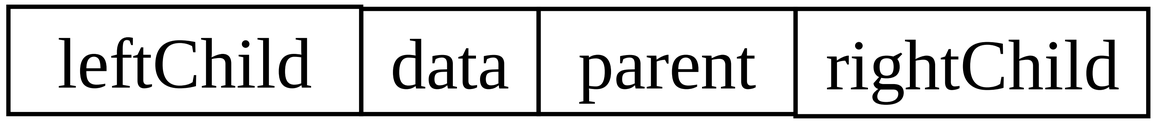
二叉树的存储表示：链表

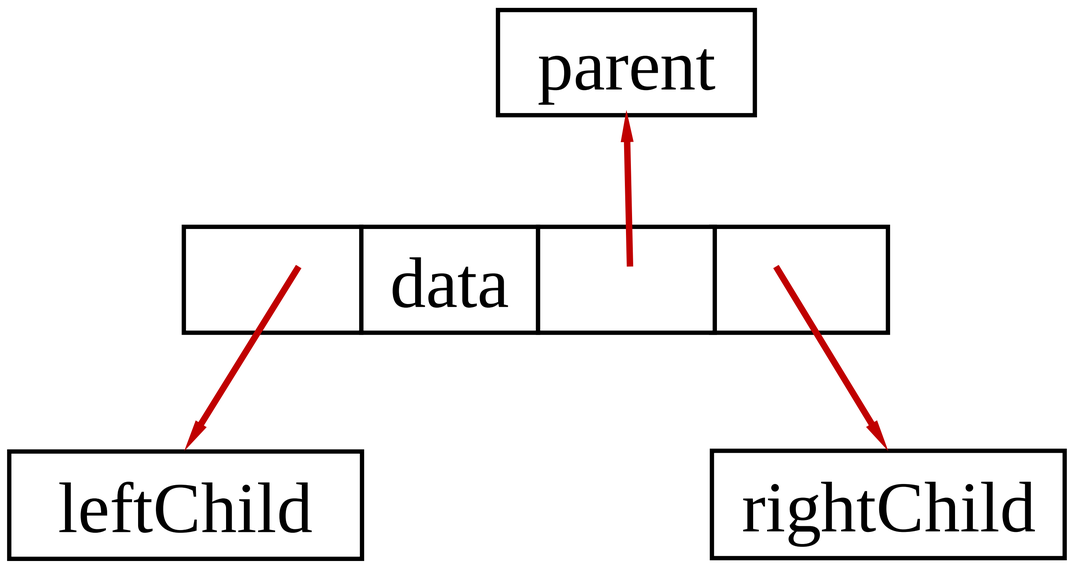
- 用链表表示二叉树时，每个结点包含3个域，一个用于存放结点本身的数据，一个用于存放指向左子结点的指针，一个用于存放指向右子结点的指针，称为二叉链表



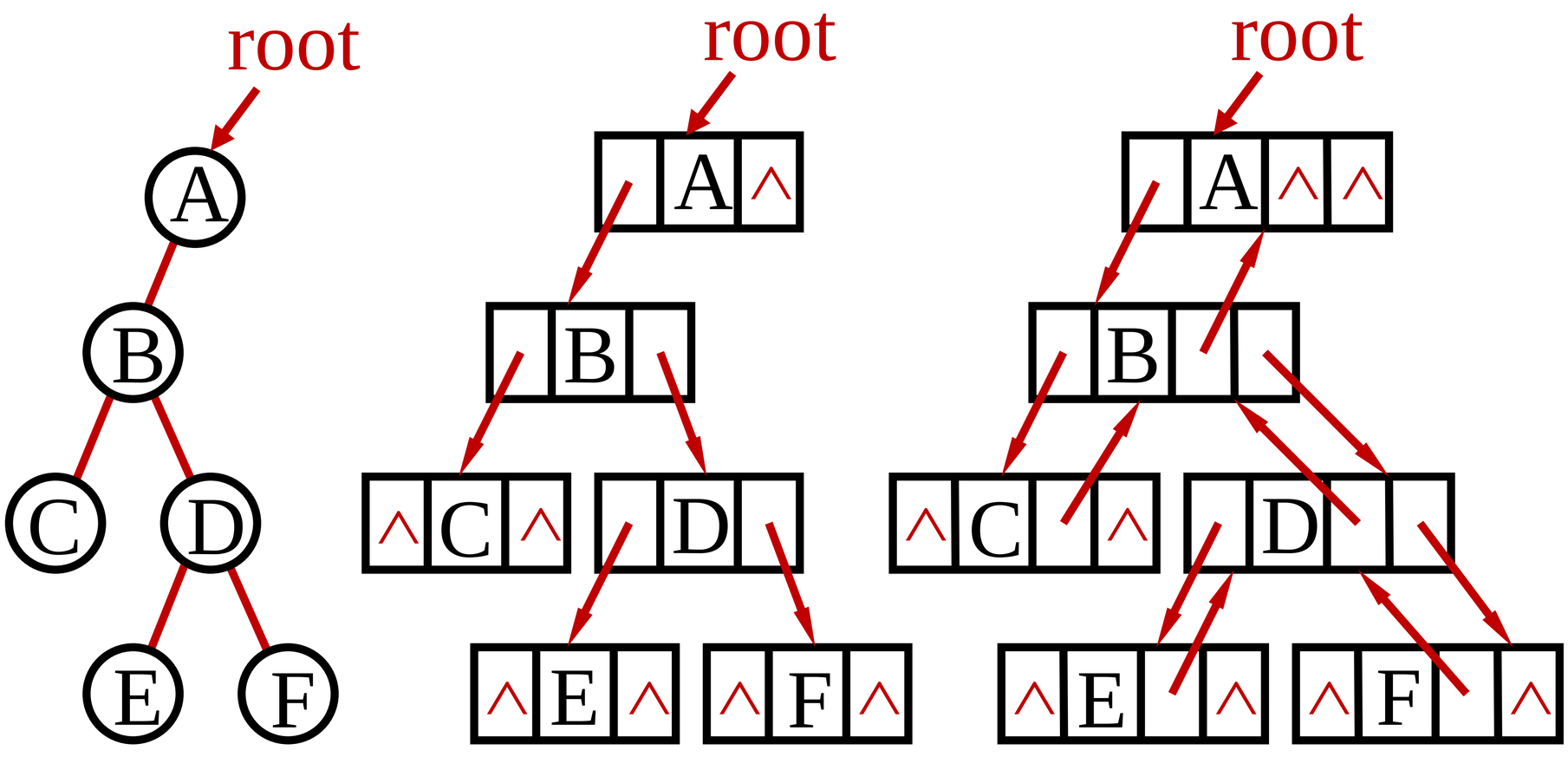
二叉树的存储表示：链表

- 二叉链表要从子结点找父结点很困难，可以为每个结点增加一个指向父结点的指针，这种链表称为**三叉链表**





二叉树链表表示示例



二叉树的类定义

```
template <class Type> class BinaryTree;
template <class Type> class BinTreeNode {
    friend class BinaryTree <Type>;
    private:
        Type data;
        BinTreeNode <Type> *leftChild, *rightChild;
    public:
        BinTreeNode( ) : leftChild(NULL), rightChild(NULL) { }
        BinTreeNode(Type d, BinTreeNode <Type> *l = NULL,
            BinTreeNode <Type> *r = NULL) : data(d), leftChild(l), rightChild(r) { }
        Type GetData( ) const { return data; }
        BinTreeNode <Type> * GetLeft( ) const { return leftChild; }
        BinTreeNode <Type> * GetRight( ) const { return rightChild; }
        void SetData (const Type &item) { data = item; }
        void SetLeft(BinTreeNode <Type> *l) { leftChild = l; }
        void SetRight(BinTreeNode <Type> *r) { rightChild = r; }
};
```

二叉树的类定义

```
template <class Type> class BinaryTree {
  protected:
    BinTreeNode <Type> *root;
    Type RefValue;
    void CreateBinTree(istream &in, BinTreeNode <Type> *subtree);
    bool Insert(BinTreeNode <Type> *subtree, const Type &x);
    void Destroy(BinTreeNode <Type> *subtree);
    BinTreeNode <Type> * Copy(BinTreeNode <Type> *origNode);
    int Height(BinTreeNode <Type> *subtree);
    int Size(BinTreeNode <Type> *subtree);
    BinTreeNode <Type> * Parent(BinTreeNode <Type> *subtree,
                                BinTreeNode <Type> *current);
```

二叉树的类定义

```
BinTreeNode * Find(BinTreeNode <Type> *subtree, const Type &x) const;
void Traverse(BinTreeNode <Type> *subtree, ostream &out);
void PreOrder(BinTreeNode <Type> *subTree,
   void (*visit) (BinTreeNode <Type> *p));
void InOrder(BinTreeNode <Type> *subTree,
   void (*visit) (BinTreeNode <Type> *p));
void PostOrder(BinTreeNode <Type> *subTree,
   void (*visit) (BinTreeNode <Type> *p));
friend istream & operator >> (istream &in, BinaryTree <Type> &Tree);
friend ostream & operator << (ostream &out, BinaryTree <Type> &Tree);
```

二叉树的类定义

```
public:
    BinayTree( ) : root(NULL);
    BinayTree(Type value) : RefValue(value), root(NULL) { }
    BinaryTree(BinaryTree <Type> &s);
    ~BinaryTree( ) { Destroy(root); }
    bool IsEmpty( ) { return root==NULL; }
    BinTreeNode <Type> * Parent(BinTreeNode <Type> *current)
    { return (root==NULL || root==current) ? NULL : Parent(root, current); }
    BinTreeNode <Type> * LeftChild(BinTreeNode <Type> *current)
    { return (current!=NULL) ? current->leftChild : NULL; }
    BinTreeNode <Type> * RightChild(BinTreeNode <Type> *current)
    { return (current!=NULL) ? current->rightChild : NULL; }
```

二叉树的类定义

```
int Height( ) { return Height(root); }
int Size( ) { return Size(root); }
BinTreeNode <Type> * GetRoot( ) const { return root; }
void PreOrder (void (*visit) (BinTreeNode <Type> *p))
{ PreOrder(root, visit); }
    void InOrder(void (*visit) (BinTreeNode <Type> *p))
    { InOrder(root, visit); }
    void PostOrder(void (*visit) (BinTreeNode <Type> *p))
    { PostOrder(root, visit); }
    void LevelOrder(void (*visit) (BinTreeNode <Type> *p));
    int Insert(const Type &item);
    BinTreeNode <Type> * Find(Type &item) const;
};
```

二叉树部分成员函数的实现

```
template <class Type> void BinaryTree <Type> ::
Destroy(BinTreeNode <Type> *subTree) {
    if (subTree!=NULL) {
       Destroy(subTree->leftChild);
       Destroy(subTree->rightChild);
       delete subTree;
    }
}
```

二叉树部分成员函数的实现

```
template <class Type> BinTreeNode <Type> * BinaryTree <Type> ::
Parent(BinTreeNode <Type> *subTree, BinTreeNode <Type> *current) {
    if (subTree==NULL)  return NULL;
    if (subTree->leftChild==current || subTree->rightChild==current)
        return subTree;
    BinTreeNode <Type> *p = Parent(subTree->leftChild, current);
    if (p!=NULL)  return p;
    else  return Parent(subTree->rightChild, current);
}
```

二叉树部分成员函数的实现

```
// 遍历并输出根为subTree的二叉树
template <class Type> void BinaryTree <Type> ::
Traverse(BinTreeNode <Type> *subTree, ostream &out) {
    if (subTree!=NULL) {
        out<<subTree->data<<‘ ’;
        Traverse(subTree->leftChild, out);
        Traverse(subTree->rightChild, out);
    }
}
template <class Type>
ostream & operator << (ostream &out, BinaryTree <Type> &Tree) {
    out<<“二叉树的前序遍历\n”;
    Tree.Traverse(Tree.root, out);
    out<<endl;
    return out;
}
```

二叉树的遍历

- **深度优先遍历**

中左右 • **前序遍历**：**首先访问根；然后按前序遍历根的左子树；最后按前序遍历根的右子树**

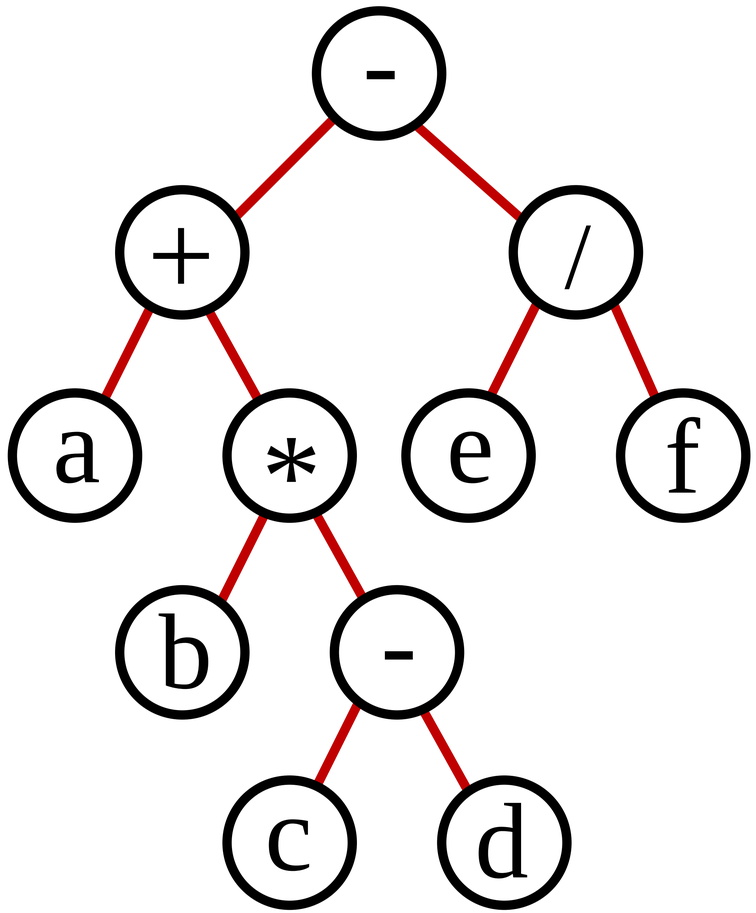
左中右 • **中序遍历**：**首先按中序遍历根的左子树；然后访问根；最后按中序遍历根的右子树**

左右中 • **后序遍历**：**首先按后序遍历根的左子树；然后按后序遍历根的右子树；最后访问根**

- **广度优先遍历**
  - **二叉树的层次遍历方法与一般有序树的几乎完全相同，不再给出**

中序遍历

- 中序遍历(Inorder Traversal)二叉树算法的框架
  - 若二叉树为空，则空操作
  - 否则，中序遍历左子树，访问根结点，中序遍历右子树



遍历结果

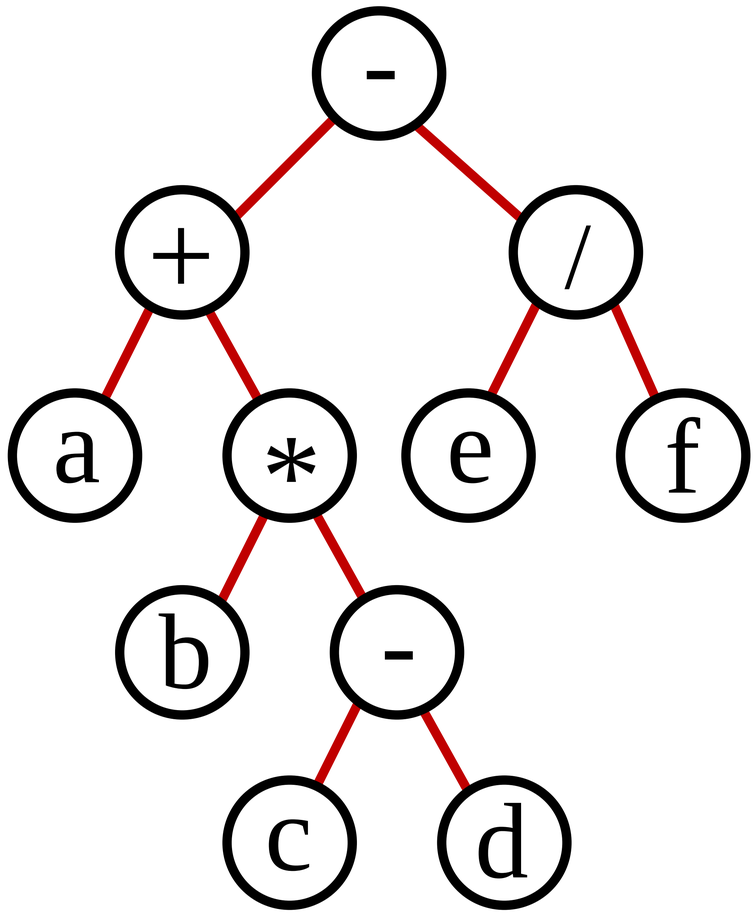
a+b*c-d-e/f

中序遍历

```
template <class Type> void BinaryTree <Type> ::
InOrder(BinTreeNode <Type> *subTree,
                void (*visit) (BinTreeNode <Type> *p)) {
    if (subTree != NULL) {
        InOrder(subTree->leftChild, visit);
        visit(subTree->data);
        InOrder(subTree->rightChild, visit);
    }
}
```

前序遍历

- **前序遍历(Preorder Traversal)二叉树算法的框架**
  - **若二叉树为空，则空操作**
  - **否则，访问根结点，前序遍历左子树，前序遍历右子树**



遍历结果

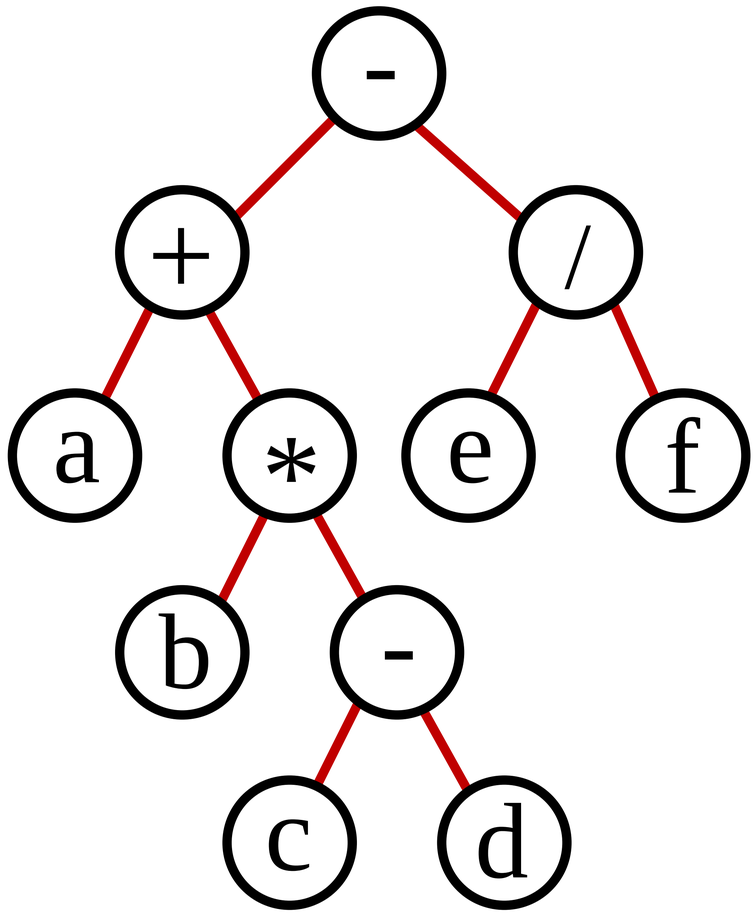
-+a*b-cd/ef

前序遍历

```
template <class Type> void BinaryTree <Type> ::
PreOrder(BinTreeNode <Type> *subTree,
                    void (*visit) (BinTreeNode <Type> *p)) {
    if (subTree!=NULL) {
        visit(subTree->data);
        PreOrder(subTree->leftChild, visit);
        PreOrder(subTree->rightChild, visit);
    }
}
```

后序遍历

- 后序遍历(Postorder Traversal)二叉树算法的框架
  - 若二叉树为空，则空操作
  - 否则，后序遍历左子树，后序遍历右子树，访问根结点



遍历结果

abcd-*+ef/-

后序遍历

```
template <class Type> void BinaryTree <Type> ::
PostOrder(BinTreeNode <Type> *subTree,
                    void (*visit) (BinTreeNode <Type> *p)) {
    if (subTree!=NULL) {
        PostOrder(subTree->leftChild, visit);
        PostOrder(subTree->rightChild, visit);
        visit(subTree->data);
    }
}
```

二叉树遍历的应用：计算结点数

- 遍历根结点的左右子树，分别计算左右子树结点个数，然后再将访问根结点的语句改为相加语句

```
template <class Type> int BinaryTree <Type> ::
Size(BinTreeNode <Type> *subTree) const {
   if (subTree==NULL)  return 0;
   else return Size(subTree->leftChild)
                    +Size(subTree->rightChild)+1;
}
```

二叉树遍历的应用：计算高度

- **如果二叉树为空，空树高度为0；否则先递归计算根结点左右子树的高度，再求出两者中最大者，并加1(增加根结点时高度加1)，得到整棵树的高度**

```
template <class Type> int BinaryTree <Type> ::
Height(BinTreeNode <Type> *subTree) const {
    if (subTree==NULL)  return 0;
    else return Max(Height(subTree->leftChild),
                    Height(subTree->rightChild))+1;
}
```

二叉树遍历的应用：复制构造

- 如果二叉树非空，则首先复制根结点，然后分别复制根结点的左右子树

```
// 公共函数：复制构造函数
template <class Type> int BinaryTree <Type> ::
BinaryTree(BinaryTree <Type> &s) {
    root = Copy(s.root);
}
```

二叉树遍历的应用：复制构造

```
// 返回一个以origNode为根的二叉树的副本
BinTreeNode <Type> * BinaryTree <Type> ::
Copy(BinTreeNode <Type> *origNode) {
    if (origNode == NULL)  return NULL;
    BinTreeNode <Type> *temp = new BinTreeNode <Type>;
    temp->data = origNode->data;  // 根结点数据
    temp->leftChild = Copy(origNode->leftChild);  // 复制左子树
    temp->rightChild=Copy(origNode->rightChild);  // 复制右子树
    return temp;  // 返回根指针
}
```

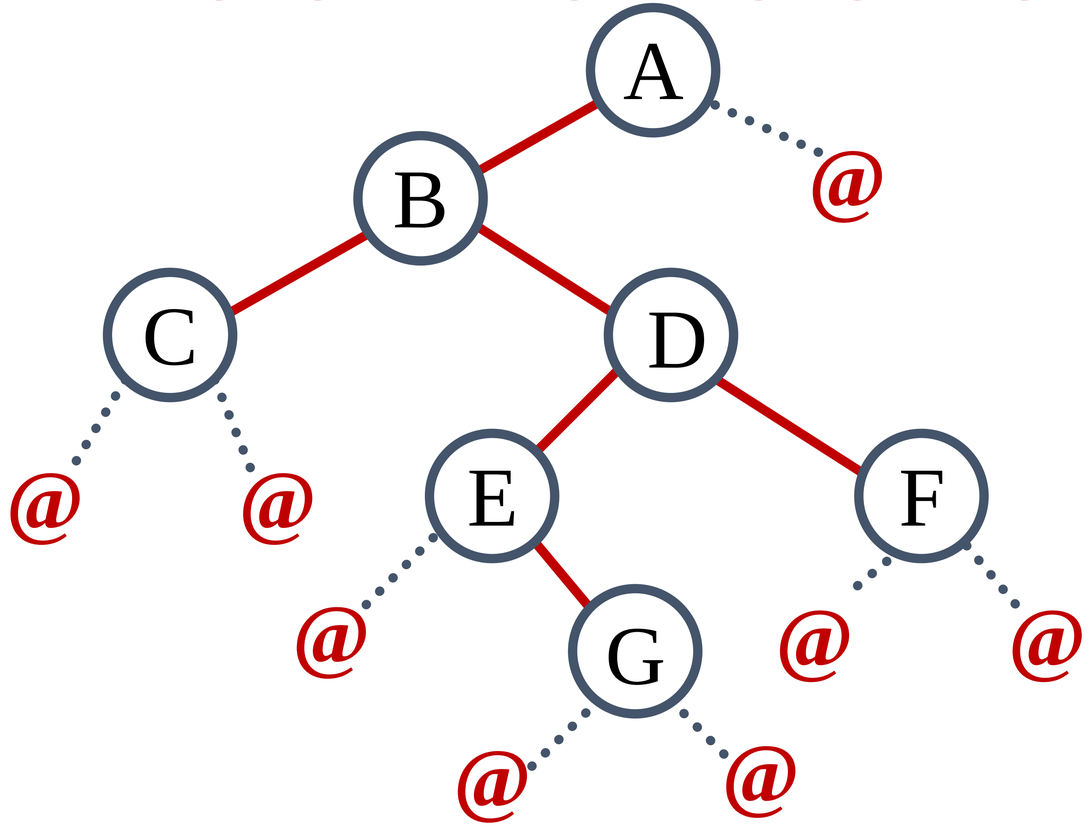
二叉树遍历的应用：判二叉树相等

```
bool Equal(BinTreeNode <Type> *a, BinTreeNode <Type> *b) {
    if (a==NULL && b==NULL)  return true;
    if (a!=NULL && b!=NULL &&
        a->data==b->data &&  // 两者根结点的数据相等
        Equal(a->leftChild, b->leftChild) &&  // 且左子树相同
        Equal(a->rightChild, b->rightChild))  // 且右子树相同
        return true;
   return false;
}
```

二叉树遍历的应用：建立二叉树

- 输入结点值的顺序必须对应二叉树结点前序遍历的顺序，并约定空结点的值以结束递归，此值在RefValue中，例如，用@或用-1表示字符序列或正整数序列的空结点

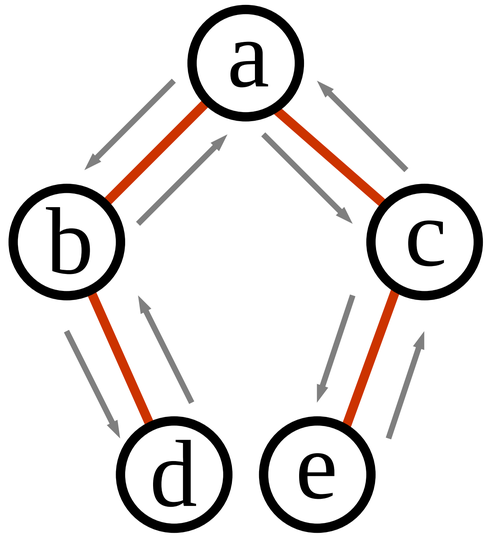
A B C @ @ D E @ G @ @ F @ @ @

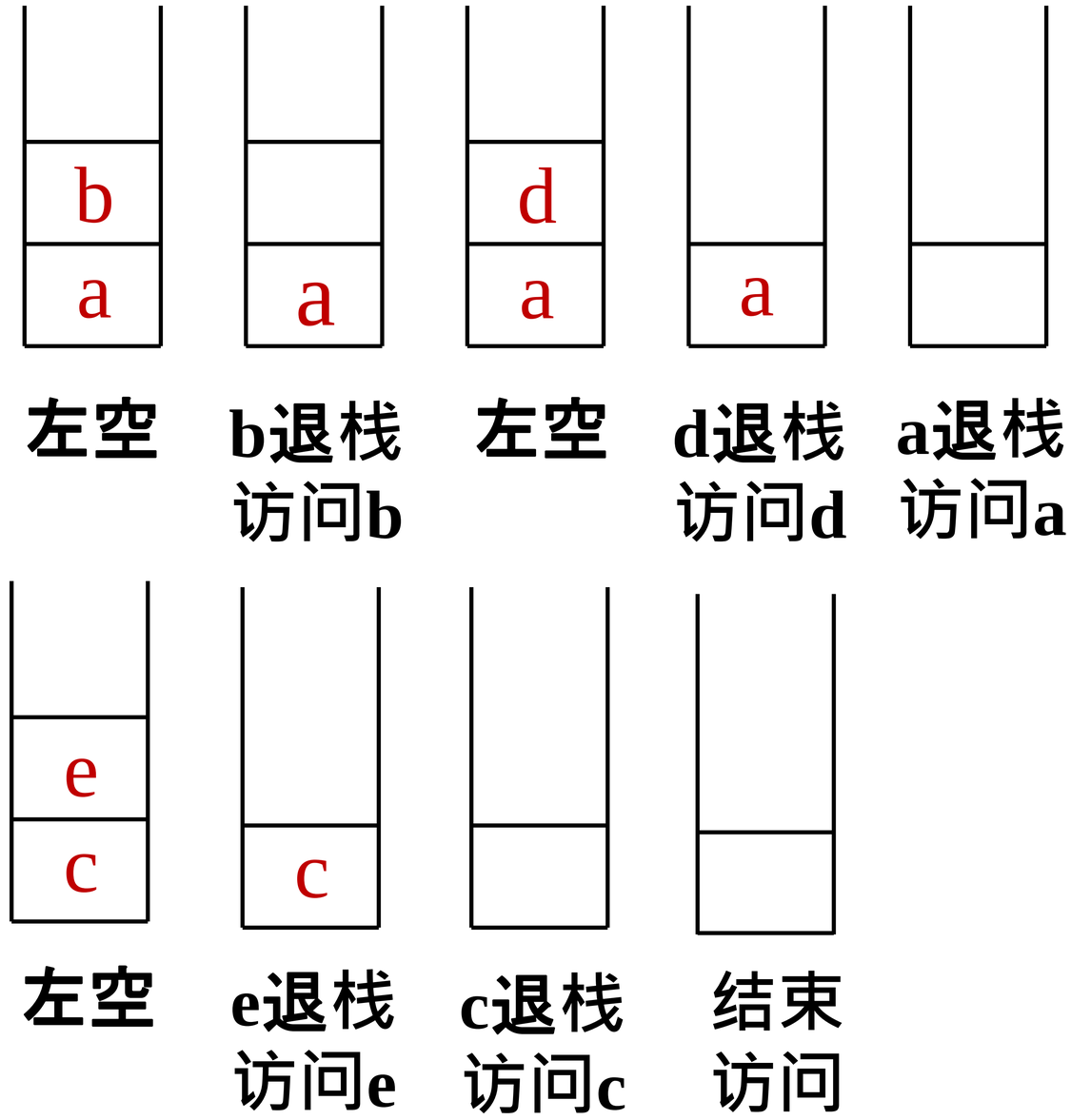


二叉树遍历的应用：建立二叉树

```
template <class Type> void BinaryTree <Type> ::
CreateBinTree(istream &in, BinTreeNode <Type> *subTree) {
    Type item;
    if (!in.eof( )) {
        in>>item;  // 读入根结点的值
        if (item!=RefValue) {
            subTree=new BinTreeNode <Type> (item); // 建立根结点
            CreateBinTree(in, subTree->leftChild);
            CreateBinTree(in, subTree->rightChild);
        } else subTree=NULL;  // 封闭叶结点
    }
}
```

中序遍历的非递归算法

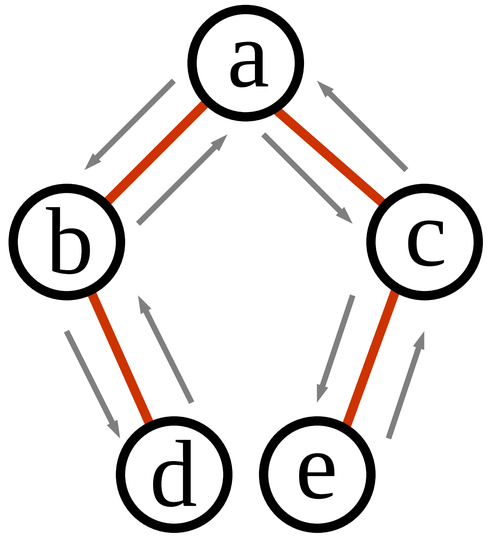

a
b
c
d
e

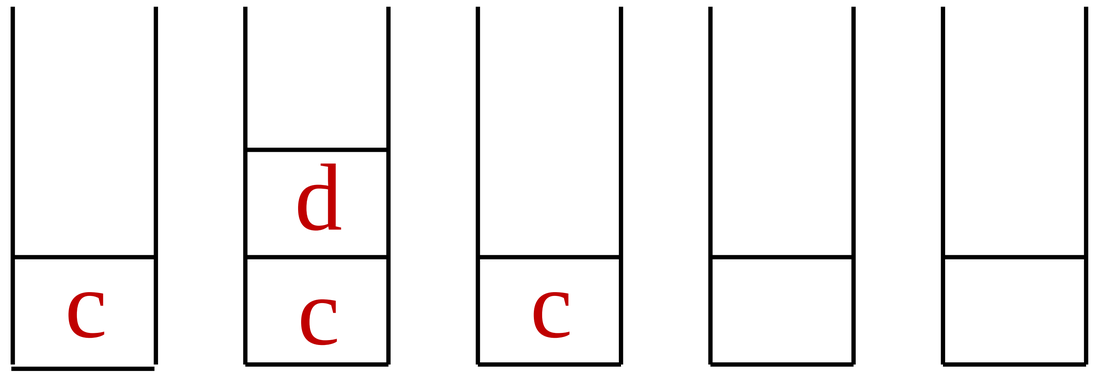

b
a
a
d
a
a
左空
b退栈
访问b
左空
d退栈
访问d
a退栈
访问a
e
c
c
左空
e退栈
访问e
c退栈
访问c
结束
访问

中序遍历的非递归算法

```
template <class Type> void BinaryTree <Type> ::
InOrder(void (*visit) BinTreeNode <Type> *p) {
    Stack <BinTreeNode <Type> *> S;
    BinTreeNode <Type> *p = root;  // 初始化
    do {
        while (p != NULL) { S.Push(p);  p = p->leftChild; } //左下到底
        if (!S.IsEmpty( )) {  // 栈非空
            S.Pop(p);  // 退栈
            visit(p);  // 访问根
            p = p->rightChild;  // 向右链走
        }
    } while (p != NULL || !S.IsEmpty( ));
}
```

前序遍历的非递归算法





访问	访问	退栈	退栈	访问
a	b	d	c	e
进栈	进栈	访问	访问	左进
c	d	d	c	空
左进	左进	左进	左进	栈空
b	空	空	e	结束

前序遍历的非递归算法

```
template <class Type> void BinaryTree <Type> ::
PreOrder(void (*visit) BinTreeNode <Type> *p) {
    Stack <BinTreeNode <Type> *> S;
    BinTreeNode <Type> *p = root;  // 初始化
    S.Push(NULL);
    while (p != NULL) {
        visit(p);
        // 预留右子树指针在栈中
        if (p->rightChild != NULL)  S.Push(p->rightChild);
        if (p->leftChild != NULL)  p = p->leftChild;  // 进左子树
        else  S.Pop(p);  // 左子树为空，进右子树
    }
}
```

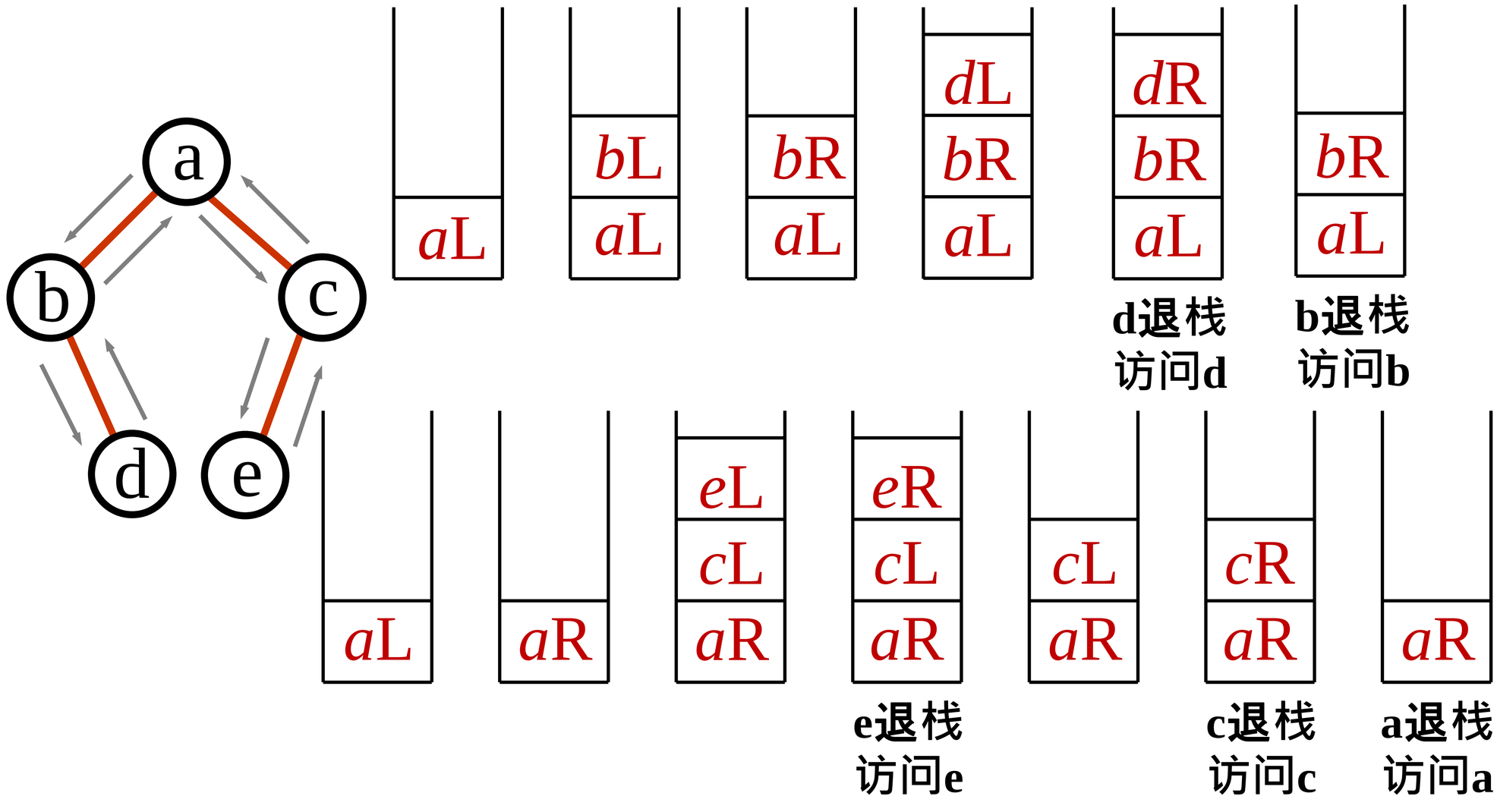
后序遍历的非递归算法

- **后序遍历时使用的栈的结点定义**

```
template <class Type> struct StkNode {
    BinTreeNode <Type> *ptr;
    enum tag {L, R};  // 结点退栈标记
    StkNode(BinTreeNode <Type> *N=NULL) : ptr(N), tag (L) { }  // 构造函数
};
```

  - **结点出栈时，tag=L，表示从左子树退出，访问右子树**
  - **结点出栈时，tag=R，表示从右子树退出，访问结点**

后序遍历的非递归算法


a
b
c
d
e
aL
bL
aL
bR
aL
dL
bR
aL
dR
bR
aL
d退栈
访问d
bR
aL
b退栈
访问b
aL
aR
eL
cL
aR
eR
cL
aR
e退栈
访问e
cL
aR
cR
aR
c退栈
访问c
aR
a退栈
访问a

后序遍历的非递归算法

```
template <class Type> void BinaryTree <Type> ::
PostOrder(void (*visit) BinTreeNode <Type> *p) {
    Stack <StkNode <Type> *> S;  stkNode <Type> w;
    BinTreeNode <Type> *p=root;
    do {
        // 向左子树走
        while (p!=NULL) { w.ptr=p;  w.tag=L;  S.Push(w); p=p->leftChild; }
        int continue=1;  // 继续循环标记
        while (continue && !S.IsEmpty( )) {
            S.Pop(w);  p=w.ptr;
            switch (w.tag) {  // 判断栈顶tag标记
                case L:  w.tag=R;  S.Push(w); continue=0; p=p->rightChild;  break; //进右子树
                case R:  visit(p);  break; }
        }
    } while (p!=NULL || !S.IsEmpty( ));
}
```

遍历时间复杂度

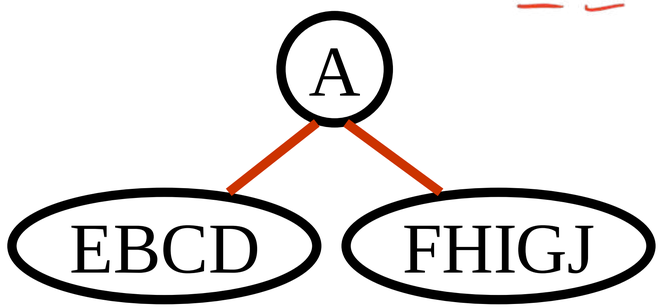
- 由于对每个结点仅访问一次，无论是否用递归实现，前序、中序、后序遍历二叉树的时间复杂度都是O(n)，空间复杂度也是O(n)
- 对前序、中序、后序来说，当二叉树的每一层都只有一个结点时，需要的栈空间最大
  - 在二叉树遍历过程中，栈的最大深度等于二叉树的高度；在同样多的结点个数下，每一层只有一个结点时，二叉树的高度为最大

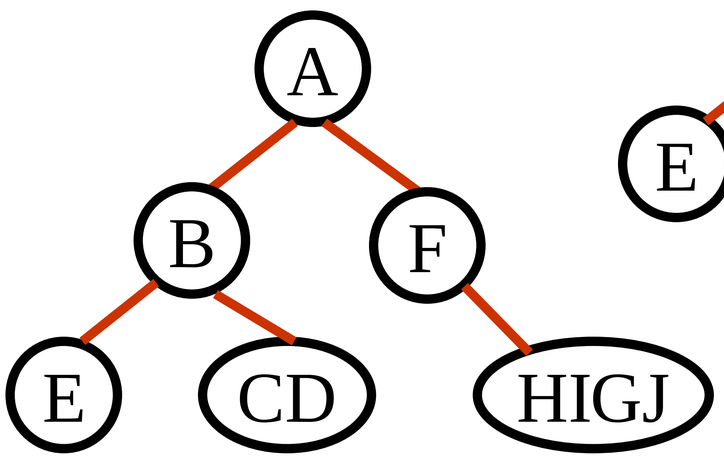
森林的遍历

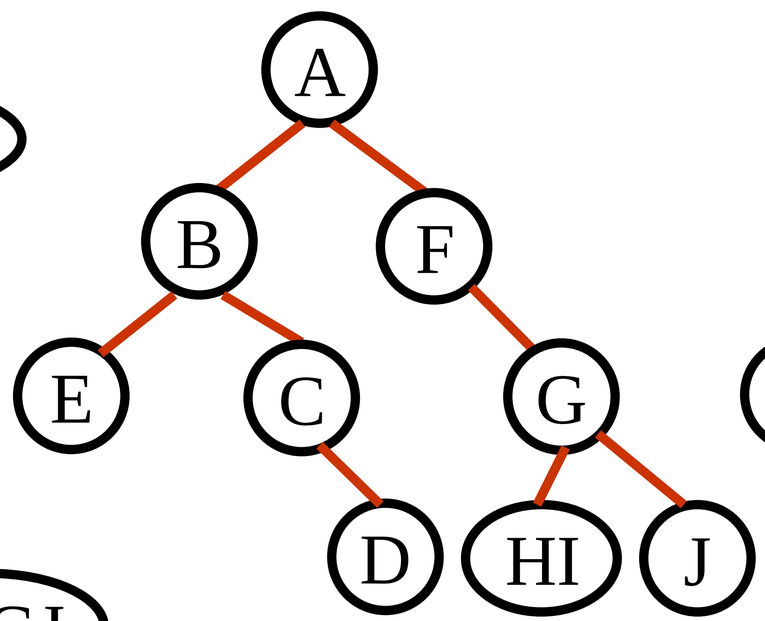
- 先根遍历森林，具体过程为
  - 如果森林非空，则访问森林中的第一棵树的根结点
  - 先根遍历第一棵树的根结点的子树森林
  - 先根遍历其它树组成的森林
- 后根遍历森林，具体过程为
  - 如果森林非空，则后根遍历第一棵树的根结点的子树森林
  - 访问森林中的第一棵树的根结点
  - 后根遍历其它树组成的森林
- 按森林转换为二叉树的方法可以证明，森林的先根、后根遍历与所转换的二叉树的先序、中序遍历的顺序相同

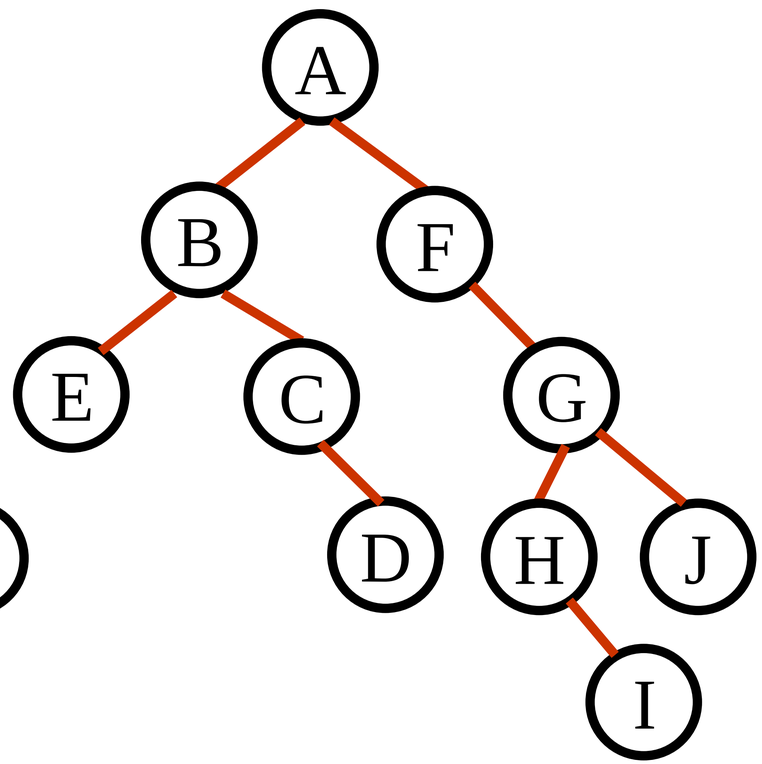
练习2

- **已知一棵二叉树先序遍历的结果ABECDFGHIJ，中序遍历的结果EBCDAFHIGJ，试画出这棵二叉树**









练习2

- 若用二叉链表作为二叉树的存储表示，编写递归算法统计二叉树中叶结点的个数

```
template <class Type> int BinaryTree <Type> ::
Leaf(BinTreeNode <Type> *subTree) const {
   if (subTree==NULL)  return 0;
   if (subTree->leftChild==NULL && subTree->rightChild==NULL) return 1;
   else return Leaf(subTree->leftChild) + Leaf(subTree->rightChild);
}
```

练习2

- 若用二叉链表作为二叉树的存储表示，编写判别给定二叉树是否为完全二叉树的算法
- 层次遍历二叉树且将空的左/右孩子结点也加入队列，当出队遇到第一个空结点时，若队列中还有元素非空，则不是完全二叉树，若队列中元素都空，则是完全二叉树

先全部加进来

本章小结

- 树
- 二叉树

作业

- 若用二叉链表作为二叉树的存储表示，编写在二叉树中查找值为x的结点并打印该结点所有祖先结点的算法(假设值为x的结点不多于1个)

- 若用二叉链表作为二叉树的存储表示，编写根据输入的广义表建立二叉树的算法
  - 广义表的表名放在表前，表示树的根结点，括号中是根的左、右子树。每个结点的左、右子树用逗号隔开。若仅有右子树没有左子树，逗号不能省略。在整个广义表表示输入的结尾加上一个特殊符号“#”，表示输入结束。例如，A(B(D, E(G,)), C(, F))#